Day 3: SOC Lab Task Report

Packet Capture and Protocol Identification with Wireshark

Field	Details
Intern Name	Amrita Pathak
Date	05 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	11:59 PM IST

Contents

1. Objective 3
2. Tools and Environment 3
3. Key Terms 3
4. Starting the Capture 4
5. Generating DNS Traffic 5
6. Generating HTTP and HTTPS Traffic 8
7. Filtering and Protocol Analysis 10
7.1 DNS filter 10
7.2 TCP filter 11
Similarly, I entered “tcp” in the filter bar to isolate all TCP traffic: 11
7.3 HTTP filter 12
Similarly, I entered “http” in the filter bar to filter HTTP traffic specifically: 12
7.4 TLS filter 14
Similarly, I entered “tls” in the filter bar to filter TLS traffic: 14
8. Protocol Behavior Summary 16
9. Findings 16
9.1 HTTP exposes everything; HTTPS hides the content but not the destination 16
9.2 TLS still leaks the destination hostname through the SNI field 16
9.3 Background application traffic outnumbers intentional traffic 16
9.4 TCP transport behavior held up consistently across every connection 17
9.5 DNS returned multiple valid answers for a single name, which is a resilience feature, not noise 17
10. Conclusion 17

1. Objective

Capture live lab traffic in Wireshark while performing a DNS lookup and visiting an HTTP test page and an HTTPS test page. Filter the capture down to DNS, TCP, and HTTP/HTTPS traffic, then identify source and destination addresses, ports, and protocol behavior for each.

2. Tools and Environment

Item	Detail
Capture tool	Wireshark 4.6.6
Capture host	Kali Linux 2026.2 (192.168.84.128)
Network interface	eth0 (VMware NAT adapter, provides real internet access)
DNS lookup tools	nslookup, dig
Browser	Firefox 140.0
HTTP test target	http://neverssl.com (deliberately never redirects to HTTPS)
HTTPS test target	https://example.com (IANA-reserved documentation domain)

Both test domains were chosen deliberately rather than a random live website. example.com is reserved by IANA specifically for documentation and testing, and neverssl.com is purpose-built to stay on plain HTTP when almost every other site now force-upgrades to HTTPS. Together, they give a clean, permission-free before/after comparison of encrypted versus unencrypted web traffic.

3. Key Terms

- DNS (Domain Name System): translates a human-readable domain name like example.com into the numeric IP address a machine actually needs to connect to it.
- UDP (User Datagram Protocol): a lightweight transport protocol with no handshake and no delivery guarantee. DNS runs over UDP because a lookup is small and fast, so the minor risk of a dropped packet is worth the speed.
- TCP (Transmission Control Protocol): a transport protocol that guarantees ordered, reliable delivery by first establishing a connection through a three-way handshake before any data is sent. HTTP and HTTPS both run on top of TCP.
- Three-way handshake: the SYN, SYN-ACK, ACK exchange that opens every TCP connection. It confirms both sides are reachable and ready before any actual data is exchanged.
- HTTP (Hypertext Transfer Protocol): the protocol web browsers use to request and receive pages. On its own it sends everything, including headers and page content, as plain readable text.
- HTTPS / TLS (Transport Layer Security): HTTP running inside an encrypted TLS session. TLS wraps the same HTTP request and response in encryption so the content is unreadable to anyone capturing the traffic in transit.
- SNI (Server Name Indication): a field inside the TLS handshake that names the destination hostname. It is sent before encryption is active, so it stays readable even on an otherwise encrypted connection.
- OCSP (Online Certificate Status Protocol): the check a browser makes to confirm a website's HTTPS certificate has not been revoked. It runs as a separate request, over plain HTTP, even when it is checking on behalf of an HTTPS connection.

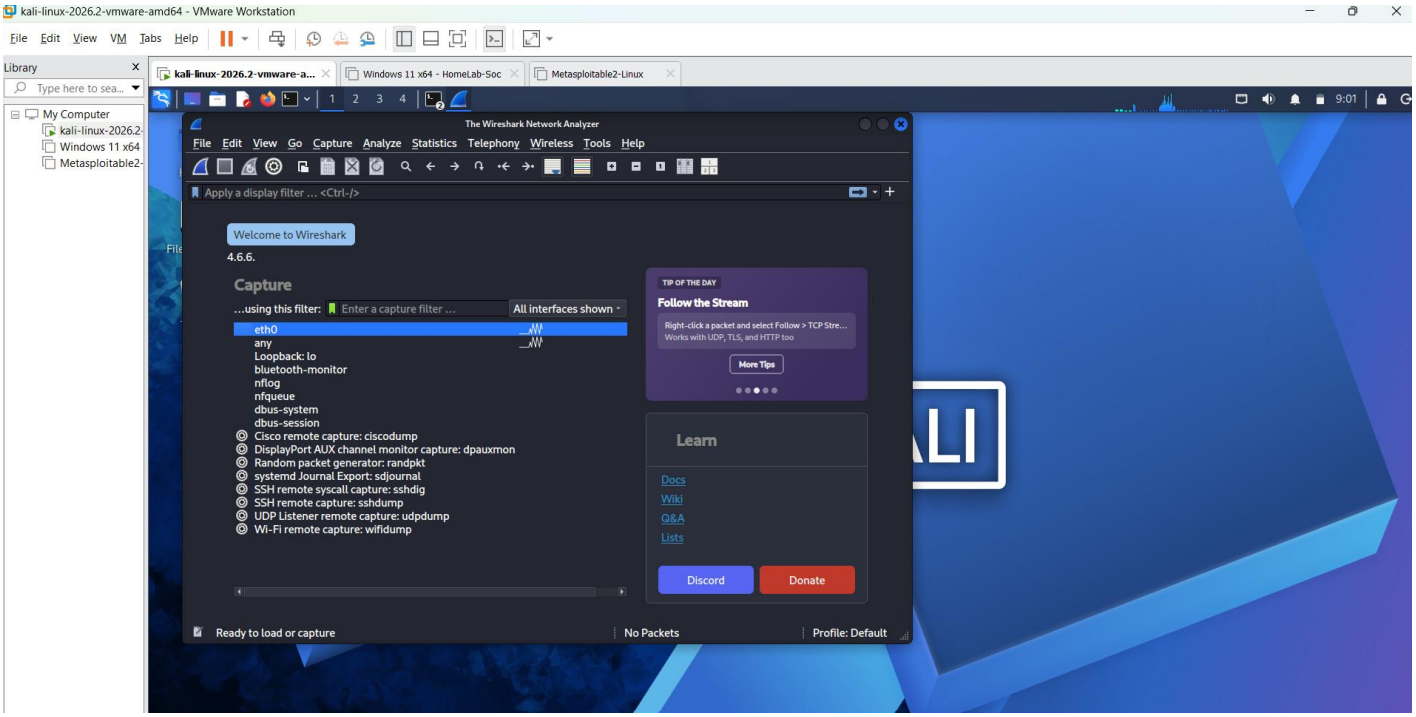
4. Starting the Capture

Wireshark needs elevated privileges to capture raw traffic off a network interface, so it was launched from a terminal with sudo rather than from a desktop icon.

```
sudo wireshark
```

Wireshark logs its own startup activity directly in the terminal it was launched from, even though the actual capture window opens separately as its own GUI. The two GUI warning lines about a host portal are a known Linux desktop-integration quirk and have no effect on the capture itself. Once launched, Wireshark opened to its interface list, prompting a choice of which network adapter to capture from.

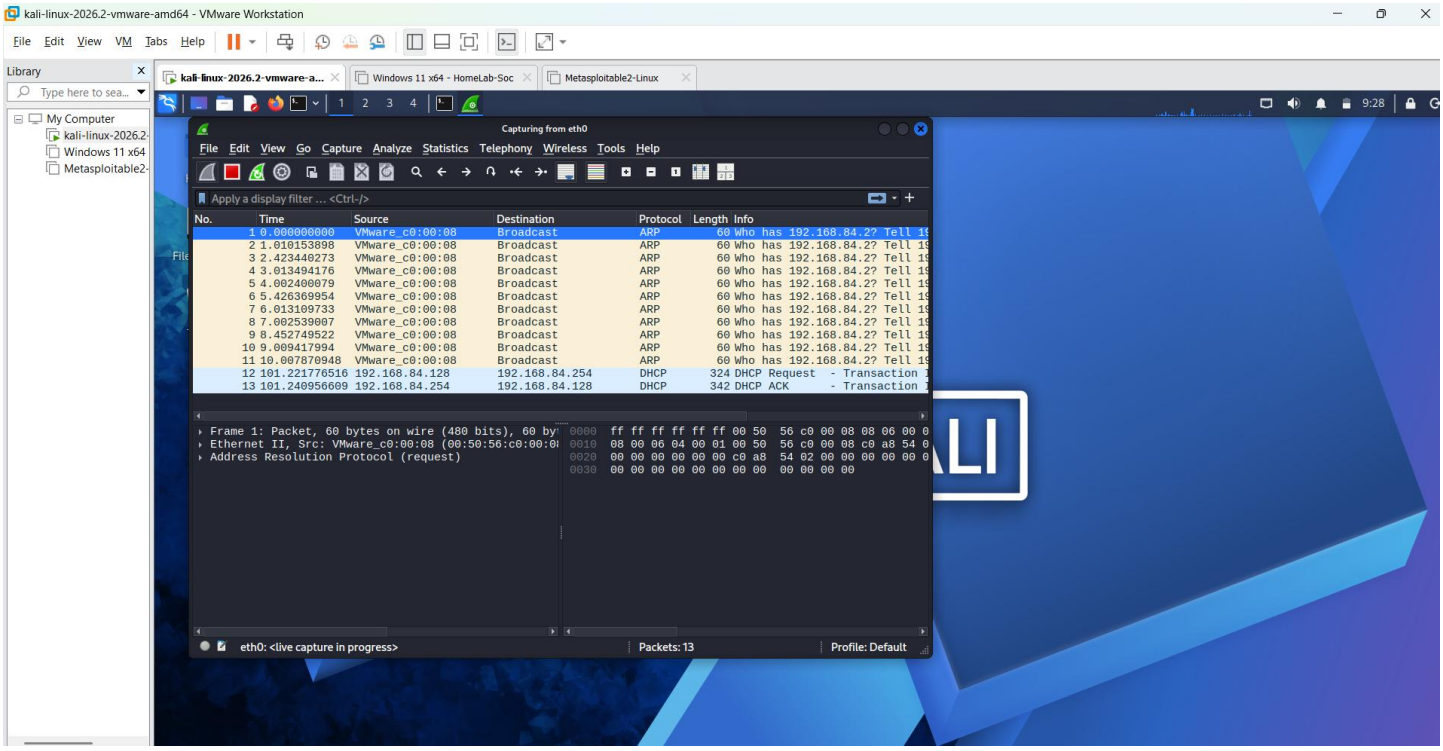
This was launched with elevated privileges and pointed at the eth0 interface, since this is the adapter carrying real traffic to and from the internet through the VMware NAT connection, as opposed to the loopback or virtual monitoring interfaces also listed.



Wireshark's interface list. eth0 was selected, visible already showing live traffic activity before the capture even started.

Once eth0 was selected, Wireshark began recording every packet flowing through that interface continuously, with no fixed duration. It kept running in the background while the DNS lookups and browser visits below were performed, exactly as a network tap would in a real monitoring setup.

Notice the phrase – “live capture in progress” at the bottom. Currently, the packets are set to 13 and go all the way up to 8785 until stopped when all the packets are captured in the steps below.

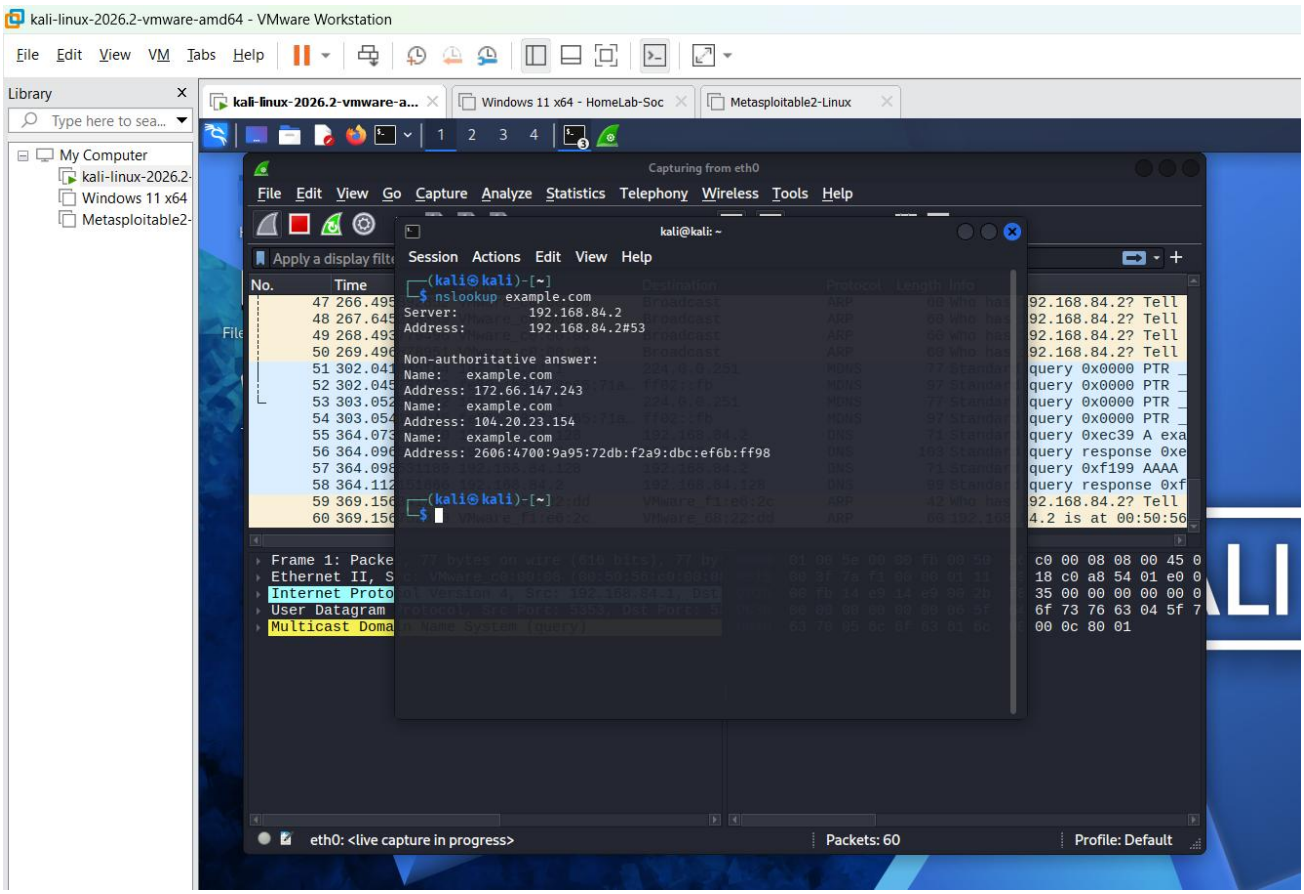


Capture running live. The status bar shows "eth0: live capture in progress" and a packet count of 13, climbing continuously as ARP and DHCP background traffic is recorded before any deliberate action is taken.

5. Generating DNS Traffic

Two different DNS tools were used deliberately, both querying the same domain, so the underlying protocol exchange could be compared against two different presentations of the same result.

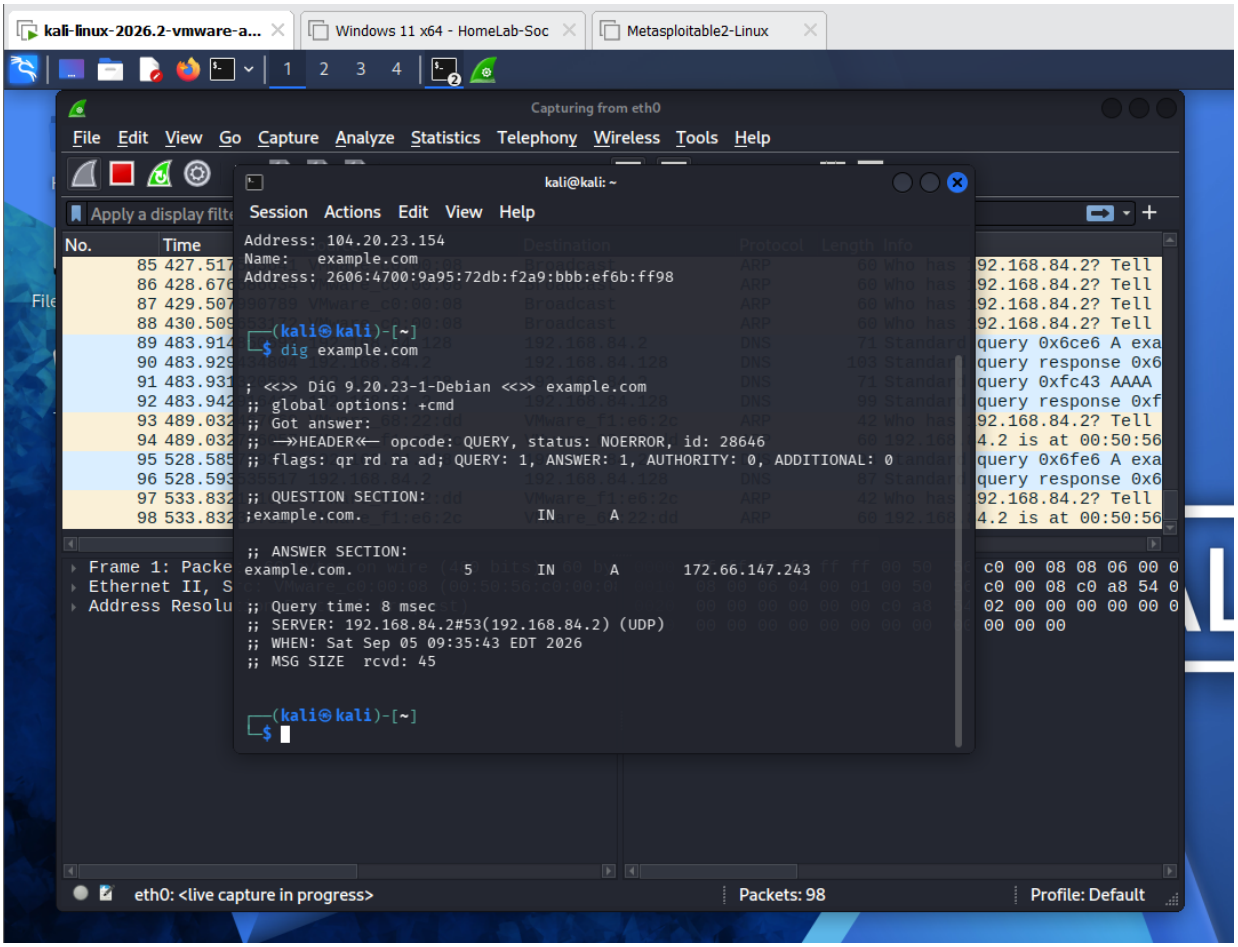
`nslookup example.com`



nslookup result: example.com resolved to two IPv4 addresses (172.66.147.243 and 104.20.23.154) and one IPv6 address, via the DNS server at 192.168.84.2.

Multiple returned addresses for a single domain is normal and expected. It reflects DNS-based load balancing or anycast routing, where a large service hands back more than one valid address for resilience and traffic distribution.

dig example.com



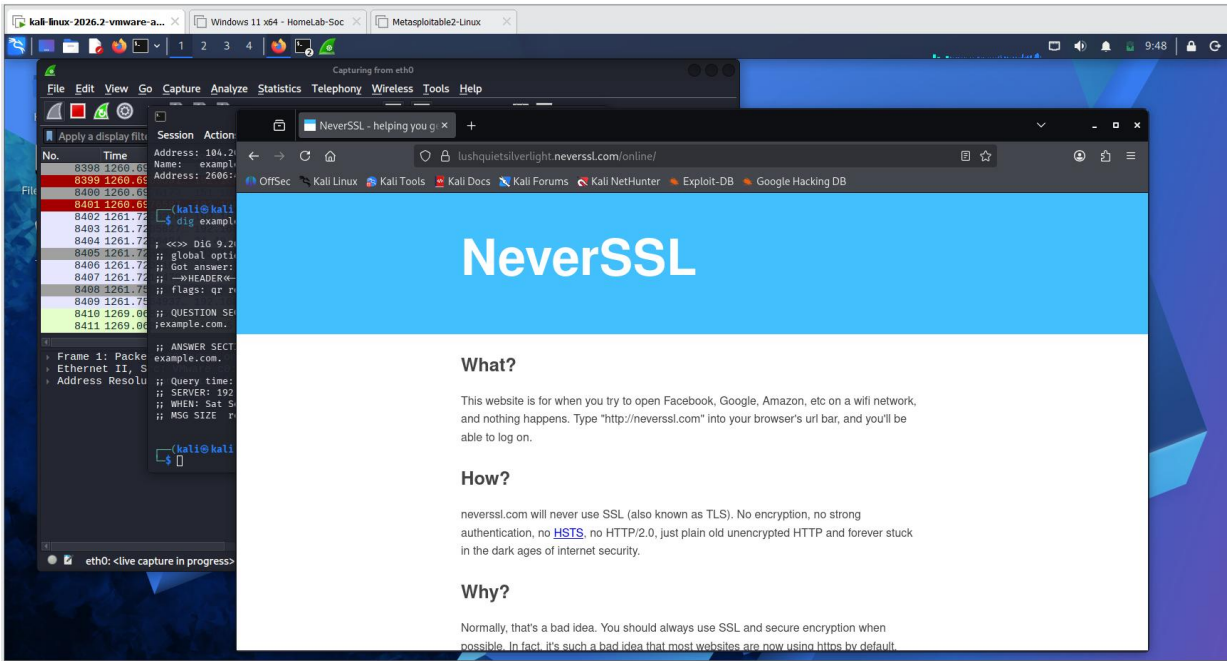
dig result for the same domain, showing the raw DNS answer section, query time, and the exact resolver (192.168.84.2#53) used.

dig exposes detail nslookup summarizes away: the query time in milliseconds, the record type (A), and the resolver's port (53) explicitly. Both tools triggered a real DNS query captured live by Wireshark in the packet list visible behind each terminal window.

6. Generating HTTP and HTTPS Traffic

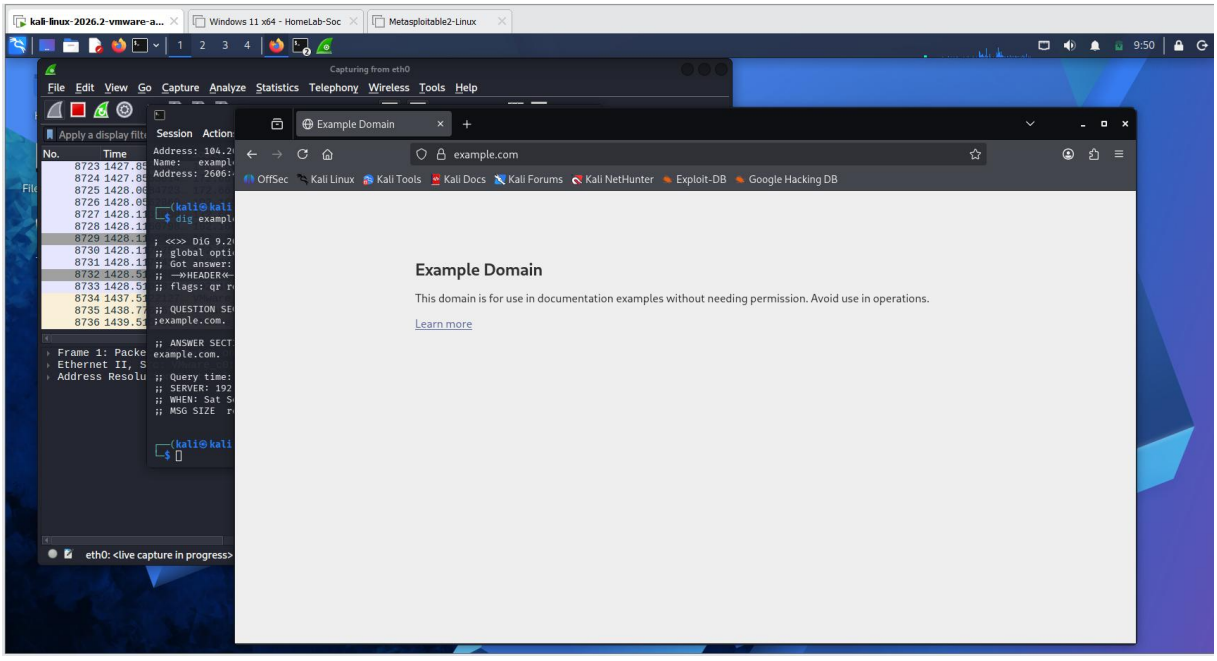
With the capture still running, Firefox was used to visit one plain HTTP page and one HTTPS page, generating the web traffic to be filtered and compared in the next section.

`http://neverssl.com`



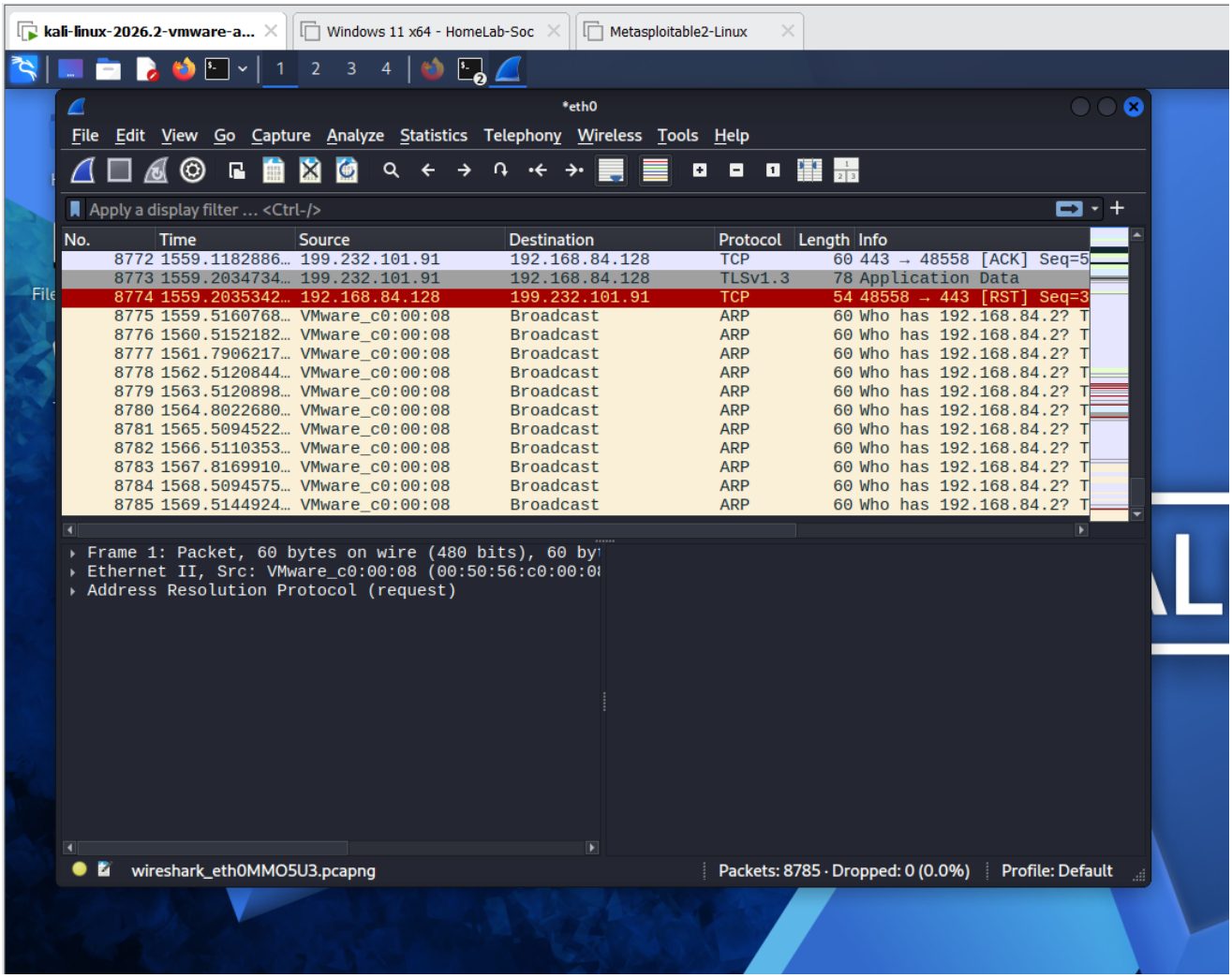
neverssl.com loaded successfully over plain HTTP. Note the address bar shows no padlock icon, since the connection is unencrypted by design.

`https://example.com`



example.com loaded over HTTPS. The padlock icon in the address bar confirms the connection is encrypted.

With DNS, HTTP, and HTTPS traffic all generated, the capture was stopped and saved for analysis.



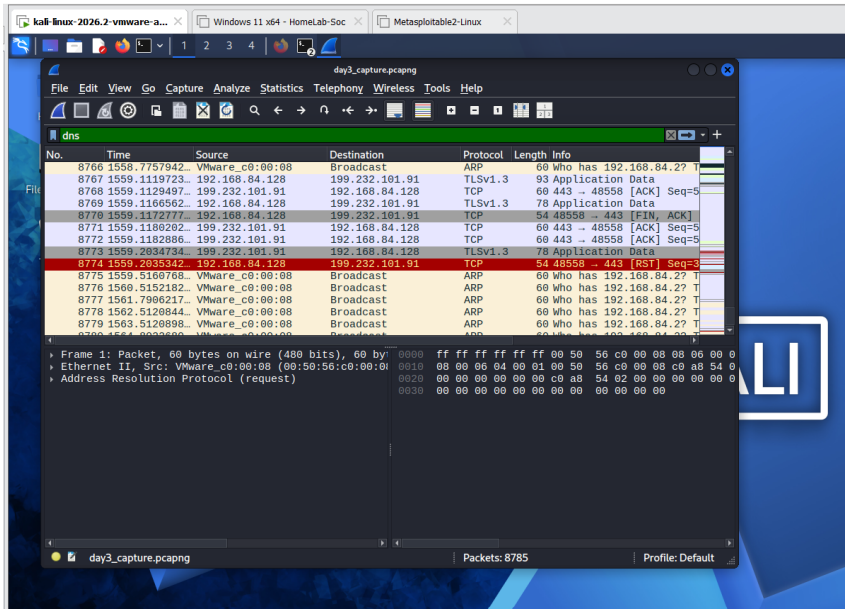
Capture stopped at 8,785 total packets and saved as day3_capture.pcapng. TLSv1.3 and Application Data entries are already visible in the unfiltered list.

7. Filtering and Protocol Analysis

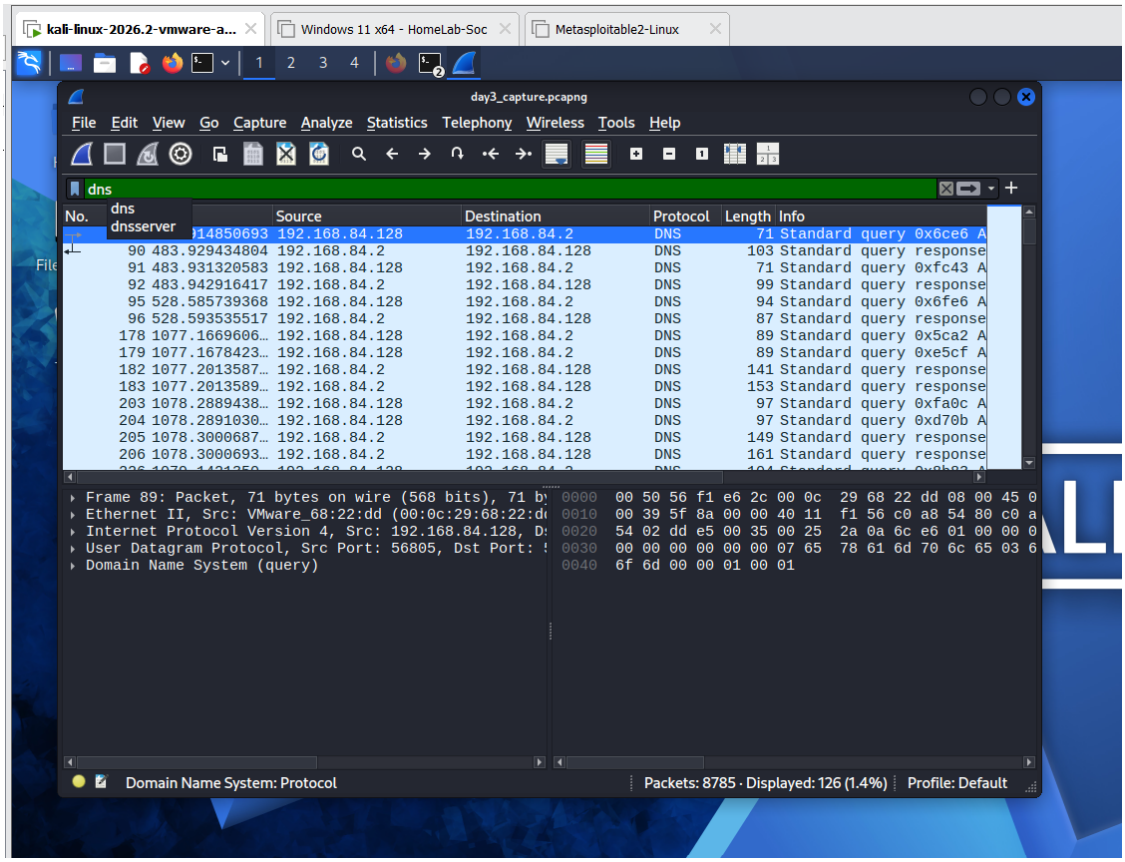
With the full capture saved, four display filters were applied in sequence to isolate each protocol of interest. Each is examined below for source, destination, ports, and protocol behavior.

7.1 DNS filter

I typed “dns” in the filter bar:



and it gave me this result:



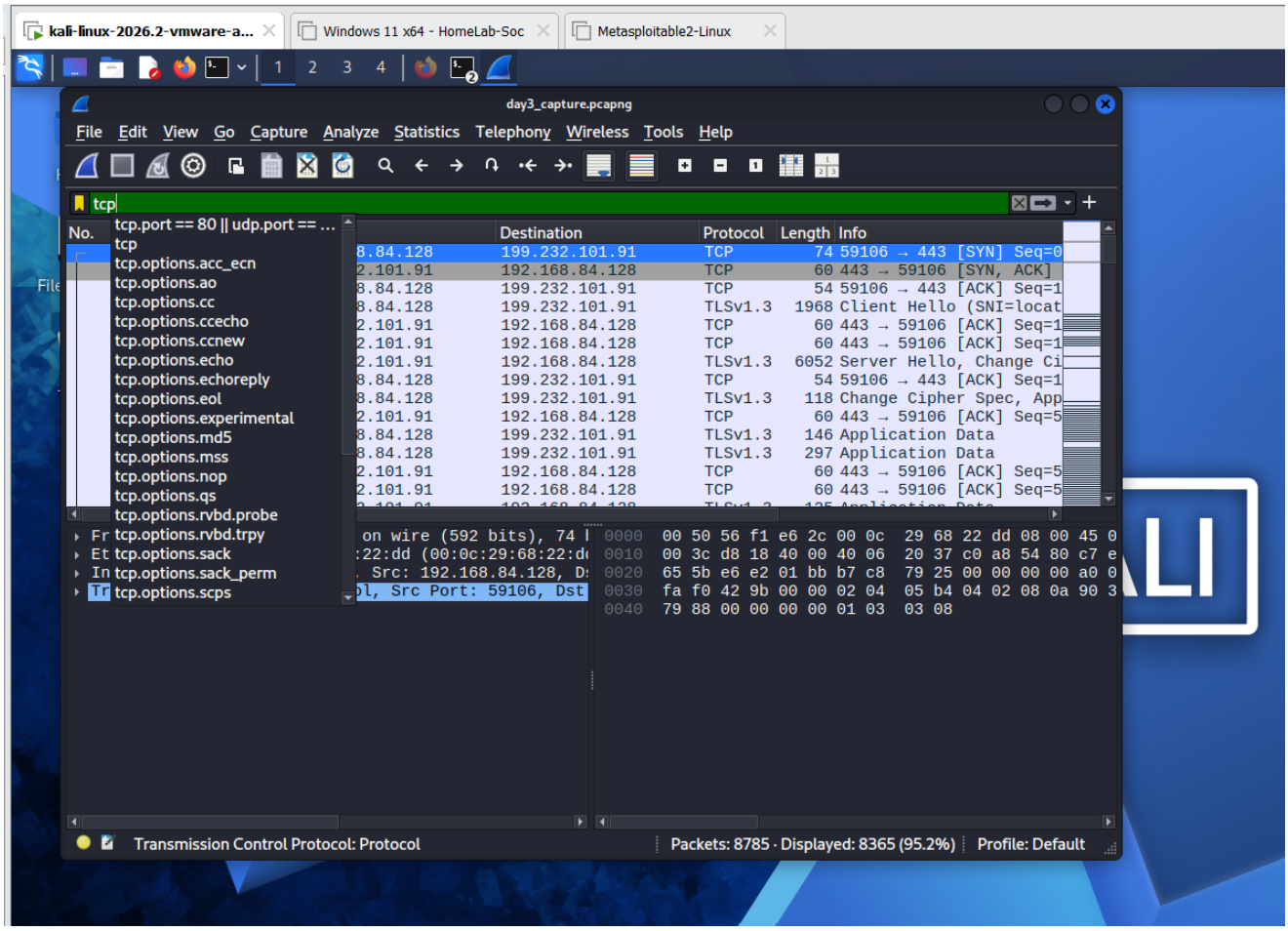
126 of 8,785 packets (1.4%) matched the DNS filter.

Source 192.168.84.128 (Kali) sent a standard query to destination 192.168.84.2 (the VMware NAT gateway acting as DNS resolver). The packet detail pane confirms this travels over UDP, with the client using a random high-numbered source port (56805 in this capture) while the server listens on the well-known destination port 53. Each query packet is answered by a matching response packet carrying the resolved address, visible directly beneath it in the list.

7.2 TCP filter

Similarly, I entered “tcp” in the filter bar to isolate all TCP traffic:

```
tcp
```



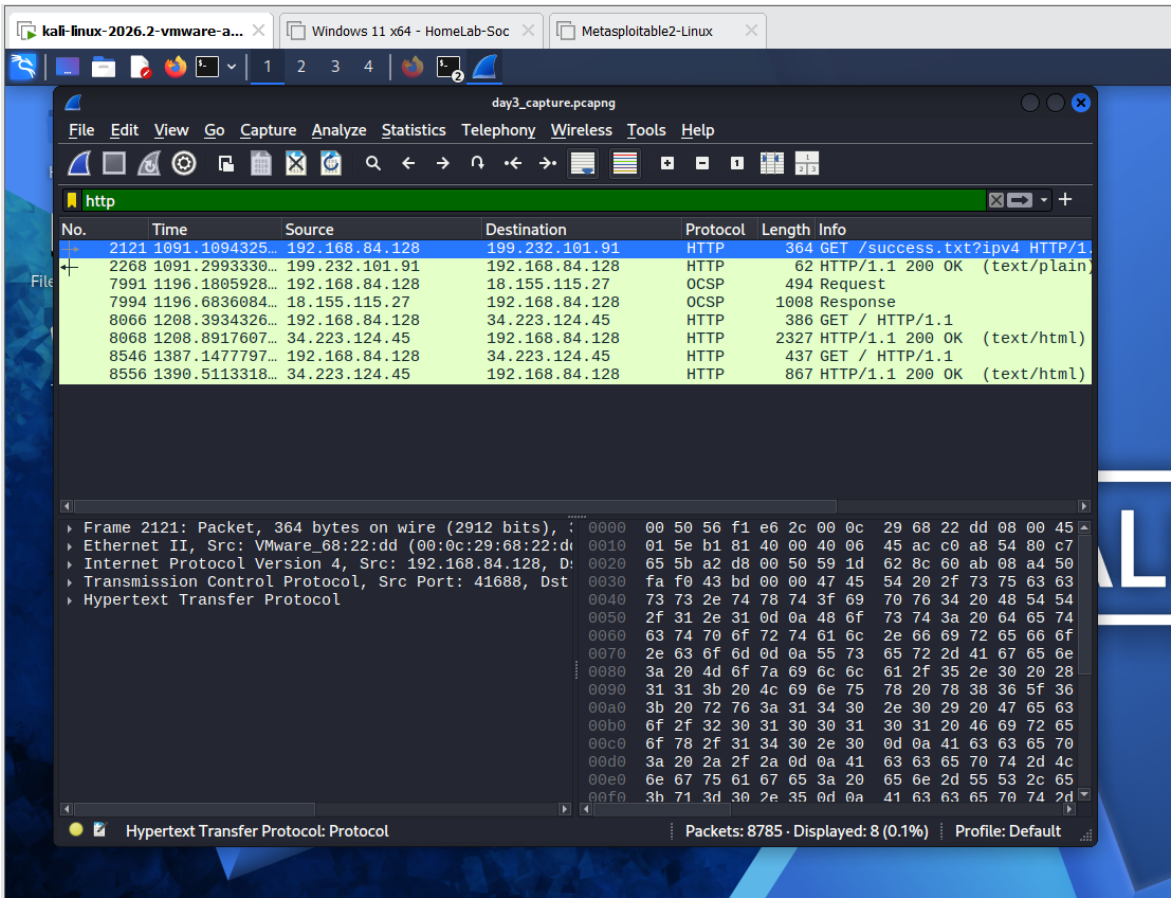
8,365 of 8,785 packets (95.2%) matched the TCP filter, the large majority of all traffic captured.

This view captures a complete TCP three-way handshake at the top of the list: a SYN from the Kali host (source port 59106) to a remote server on port 443, answered with a SYN-ACK, completed with a final ACK from the client. Immediately following the handshake, a TLS Client Hello begins, which is the first step of the encryption negotiation layered on top of the now-established TCP connection. This confirms the expected behavior: TCP builds the connection first, and only once it is established does any higher-level protocol, encrypted or not, begin exchanging data.

7.3 HTTP filter

Similarly, I entered “http” in the filter bar to filter HTTP traffic specifically:

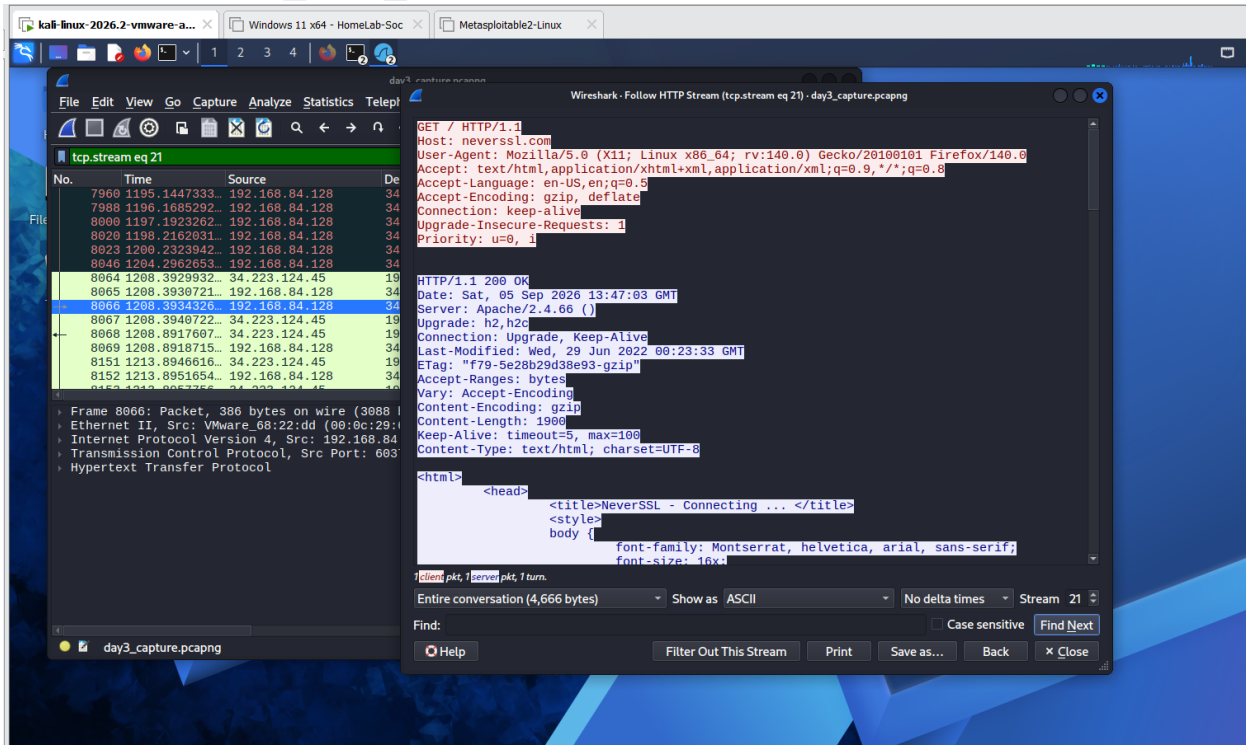
```
http
```



8 of 8,785 packets (0.1%) matched the HTTP filter.

This small set includes more than just the intended neverssl.com visit. Two extra categories of traffic appear: a GET request to /success.txt on 199.232.101.91, which is Firefox's own automatic connectivity check that runs quietly on startup before any page is manually visited, and two OCSP exchanges, the protocol browsers use to check whether a website's HTTPS certificate has been revoked. Notably, OCSP checks travel over plain HTTP even though they exist to support HTTPS connections elsewhere. The actual neverssl.com requests appear as GET / HTTP/1.1 to 34.223.124.45, each answered with 200 OK (text/html).

Right-clicking one of the neverssl.com packets and choosing Follow > HTTP Stream reveals the entire exchange in plain, human-readable text:



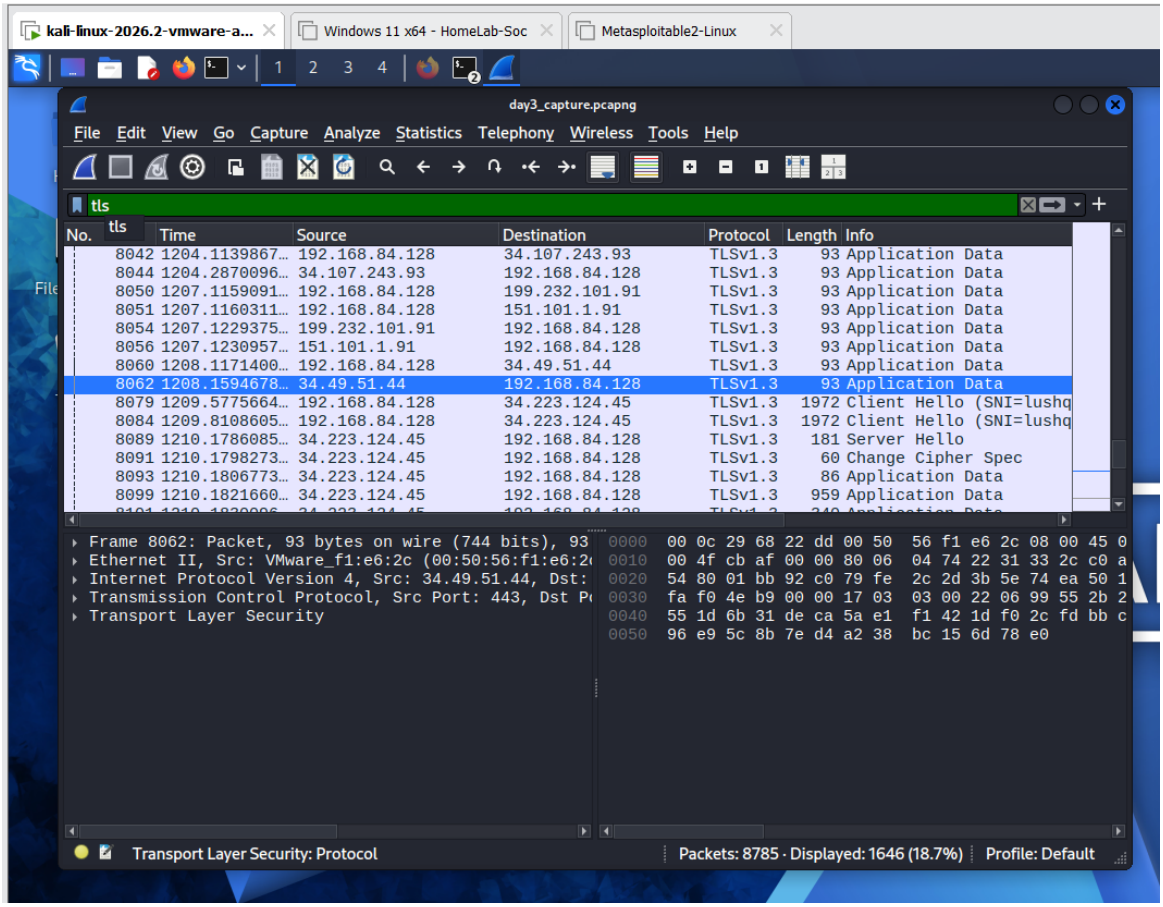
Follow HTTP Stream: the full GET request (red) and the server's response (blue) are both fully readable, including the exact page HTML returned.

Every byte of this exchange, the request headers, the browser's identity string, and the returned HTML, travels across the network as plain text. Anyone capturing this traffic on the same network path can read it directly with no decryption required. This is the plaintext half of the comparison this assignment is built around.

7.4 TLS filter

Similarly, I entered “tls” in the filter bar to filter TLS traffic:

```
tls
```



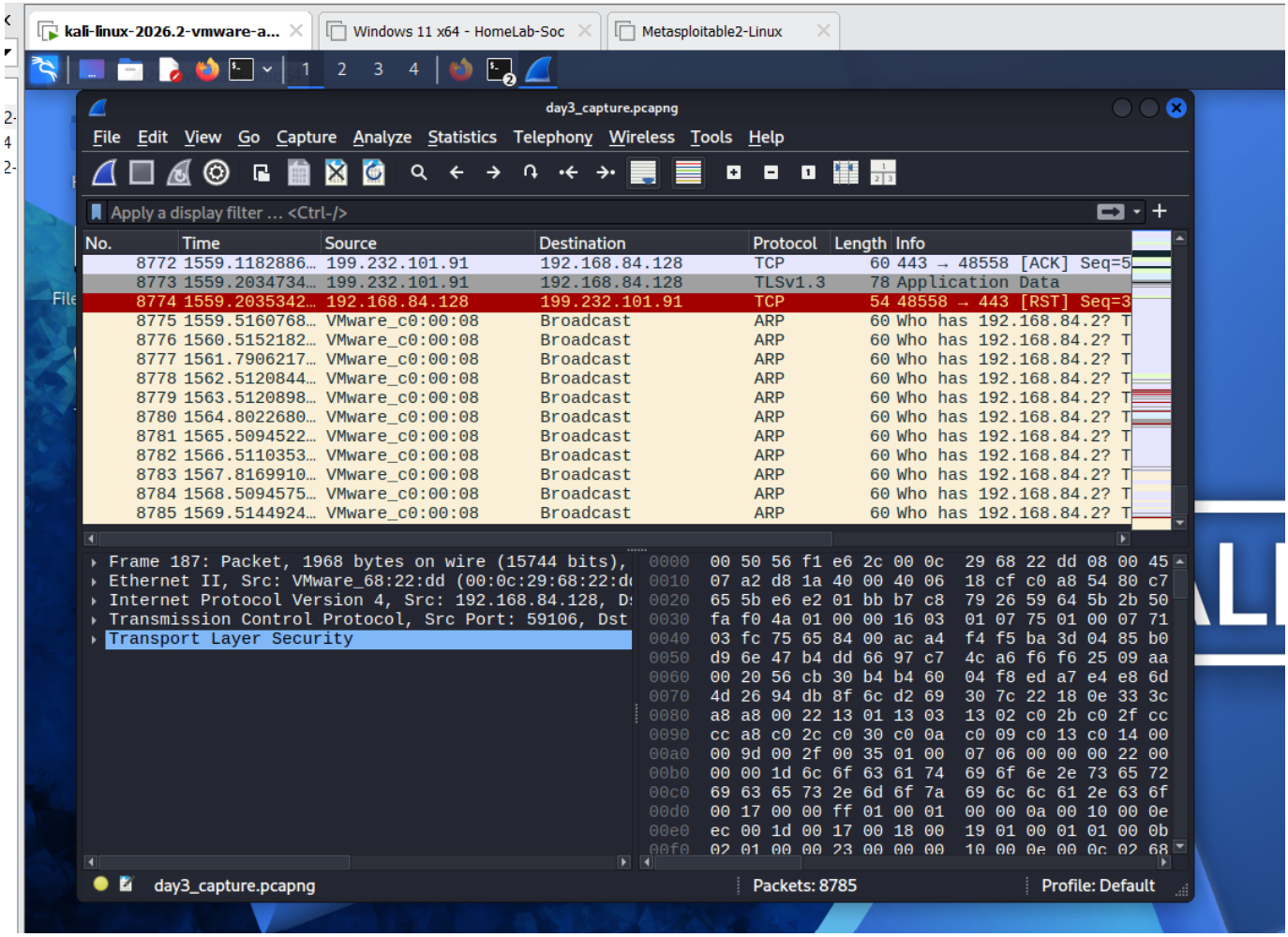
1,646 of 8,785 packets (18.7%) matched the TLS filter.

This is a meaningfully larger share of traffic than HTTP, since Firefox maintains several background HTTPS connections (telemetry, safe-browsing checks, and similar services) in addition to the deliberate visit to example.com. The handshake sequence for the example.com connection follows the expected TLS 1.3 pattern: Client Hello, Server Hello, a Change Cipher Spec record (retained in TLS 1.3 purely for compatibility with older network middleboxes rather than for any real cryptographic function), followed by Application Data once the encrypted channel is established.

One detail worth flagging specifically: even though TLS encrypts the bulk of the handshake and all application data that follows, the Client Hello's SNI (Server Name Indication) field is sent in cleartext by default. This means an observer capturing this traffic can still see which hostname a connection is destined for, even though the actual page content stays fully hidden. This is a real and often underappreciated limitation of HTTPS that is worth noting in any protocol-behavior analysis.

Attempting to Follow > TLS Stream on this traffic, as was done successfully for HTTP above, correctly failed to reassemble readable content, which is expected: TLS 1.3's encryption is precisely what should prevent this. Selecting an individual TLS packet directly and inspecting its raw bytes gives the same result more reliably.

Finally, my Wireshark terminal looks like this:



Raw hex/ASCII view of a TLS Application Data packet. The bytes shown are unstructured and non-readable, in direct contrast to the plaintext HTTP stream above.

8. Protocol Behavior Summary

Protocol	Transport	Port(s)	Encrypted?	Behavior observed
DNS	UDP	53	No	Query/response pair; multiple A records returned for load balancing
TCP	—	443 (seen), 80 (seen)	N/A (transport layer)	Three-way handshake (SYN, SYN-ACK, ACK) precedes all higher-layer data
HTTP	TCP	80	No	Fully readable request/response, including page content, in Follow Stream
TLS / HTTPS	TCP	443	Yes	Handshake visible (Client Hello/Server Hello, but Application Data unreadable)
OCSP	HTTP (TCP)	80	No	Certificate-revocation check for an HTTPS connection, sent over plain HTTP

9. Findings

9.1 HTTP exposes everything; HTTPS hides the content but not the destination

The clearest finding from this capture is the direct contrast between sections 7.3 and 7.4. The HTTP stream to neverssl.com was fully readable start to finish: the exact GET request line, every header including the browser's User-Agent string, and the full HTML of the response all appeared as plain text the moment the stream was followed. The TLS traffic to example.com, by contrast, showed only the handshake in the clear (Client Hello, Server Hello, Change Cipher Spec) before dropping into Application Data records that carry no readable structure at all, confirmed directly in the raw hex view in section 7.4.

This is not a small difference in degree, it is a difference in kind. On the HTTP connection, anyone with a packet capture on the same network path has the same visibility into the exchange as the browser itself. On the TLS connection, that same observer can tell that a connection happened, roughly how much data moved, and (as the next finding covers) which hostname it was destined for, but nothing about what was actually said.

9.2 TLS still leaks the destination hostname through the SNI field

Even though TLS 1.3 encrypts the handshake far more thoroughly than older TLS versions did, the Client Hello's SNI field is still sent in cleartext by default, because the server needs to know which hostname's certificate to present before the encrypted channel exists. This was visible directly in the TCP-filtered view in section 7.2, where a Client Hello packet's Info column named the destination hostname in the open before any encryption had been negotiated.

In practice, this means an organization's firewall, ISP, or any observer on the path can still build a list of every hostname a device has connected to over HTTPS, even without breaking the encryption itself. This is a genuine, known limitation of TLS rather than a misconfiguration in this lab. Newer extensions such as Encrypted Client Hello (ECH) are designed specifically to close this gap by encrypting the SNI field too, but ECH is not yet the default behavior most servers and browsers use, which is exactly why the plain SNI was visible in this capture.

9.3 Background application traffic outnumbers intentional traffic

Of the 8 HTTP packets captured, only a portion were the direct result of the neverssl.com visit. The rest came from Firefox's own automatic connectivity check to 199.232.101.91 and two OCSP certificate-validation exchanges, none of which were

manually triggered. The same pattern showed up at a larger scale in the TLS filter: 1,646 packets matched, far more than a single page visit to example.com would produce on its own, because Firefox holds open several background HTTPS connections (telemetry, safe-browsing list updates, and similar services) regardless of what the user is actively browsing.

This matters beyond just this lab exercise. In any real packet-capture investigation, a captured session almost always contains far more automated background chatter than the specific action being investigated. Separating deliberate, user-driven traffic from a device's own routine background noise is a core analytical skill, not an afterthought, and this capture provided a clean, low-stakes example of exactly that separation.

9.4 TCP transport behavior held up consistently across every connection

Every HTTP and TLS exchange captured was preceded by a complete three-way handshake (SYN, SYN-ACK, ACK) on the relevant destination port before any application data moved, and every connection observed used a random, unprivileged source port on the Kali host (in the 40000 to 60000 range, such as 59106) while connecting to the standard destination ports 80 or 443 on the far end. This asymmetry, an ephemeral, unpredictable port on the client side against a fixed, well-known port on the server side, is exactly how TCP client-server connections are expected to behave, and no deviation from it (retransmissions, resets appearing before a handshake is completed, or out-of-order application data) was observed anywhere in this capture.

The one RST (connection reset) packet visible in the full capture appeared only at the natural end of a completed exchange, which is normal, closing behavior rather than a sign of a dropped or interrupted connection.

9.5 DNS returned multiple valid answers for a single name, which is a resilience feature, not noise

Both nslookup and dig returned two separate IPv4 addresses and one IPv6 address for the single query example.com. This is not a fluke or an inconsistency between the two tools, since both queried the same resolver and received the same answer set. It reflects DNS-based load balancing or anycast, where a large service intentionally publishes more than one valid address so that traffic can be spread across servers and so the domain keeps resolving even if one address becomes unreachable. Recognizing this pattern matters for analysis: a domain resolving to multiple IPs across separate lookups is expected behavior for major services, not evidence of anything unusual happening on the network.

10. Conclusion

A single continuous Wireshark capture on the Kali lab VM recorded DNS, TCP, HTTP, and TLS traffic generated by nslookup, dig, and two deliberately contrasting web page visits. Filtering isolated each protocol cleanly, and direct inspection, particularly the Follow Stream comparison between the plaintext HTTP exchange and the encrypted TLS exchange, demonstrated concretely why HTTPS is necessary for protecting data in transit.

The five findings above go a step further than that basic contrast: they show a real limitation of TLS (the SNI field still leaking the destination hostname), a practical investigative skill (telling deliberate traffic apart from a device's own background chatter), consistent transport-layer behavior across every connection observed, and a correct reading of DNS returning multiple valid addresses as a resilience feature rather than an anomaly.

Together, this exercise covered the full workflow of packet-level analysis: generating known traffic, capturing it without interference, filtering by protocol, and reading the resulting behavior with the same scrutiny a SOC analyst would apply to unknown traffic on a live network.